MediAI

SMART PATIENT TRIAGE & EMERGENCY RISK PREDICTION SYSTEM

M4 — Smart Triage, Safety Rules & Alerts

MODULE IMPLEMENTATION SPECIFICATION

Assigned member	Raga Priya
Git branch	feature/m4-smart-triage
Baseline	Master Project Specification v2.0
Development workflow	Antigravity + shared GitHub repository + Pull Request review

Clinical safety: This is an AI-assisted decision-support prototype. It must not independently diagnose, prescribe, or replace certified clinical judgment.

Prepared for coordinated module development and integration.

1. Module overview and objectives

Implement deterministic Tier-1 red-flag detection, cross-category vital safety baselines, triage priority mapping, escalation/notification orchestration and auditable clinician override workflow. Deterministic safety rules take precedence over probabilistic ML.

2. Assigned responsibilities

- Implement all 12 clinical risk-domain red-flag filters from the master spec without weakening thresholds.
- Run safety pre-check before ML inference; any Tier-1 red flag forces Critical (P1) immediately.
- Apply the defined acuity bands, semantic colors and evaluation-time guidance.
- Dispatch assessment to the doctor queue and alert channel through adapters.
- Support doctor override only with mandatory rationale; persist original/final categories and audit event.

3. Features and functional requirements

- Hard red-flag engine for all 12 domains: infection/sepsis, respiratory, cardiac, neurological, blood pressure, diabetes, GI, trauma, pregnancy, allergy, dehydration/fluid balance, multi-system/toxic exposure.
- Cross-category vital baselines for HR, RR, SBP, DBP, SpO2, temperature and glucose.
- Priority mapping: Critical P1 (85–100) immediate; High P2 (60–84) 15–30 minutes; Moderate P3 (30–59) within 60 minutes; Low P4 (0–29) within 120 minutes.
- Immediate Critical/P1 override on any Tier-1 red flag independent of ML.
- Alerts and escalation; notification provider must be isolated behind adapter.
- Override requires rationale and creates an immutable/auditable record.

4. Technology stack used in this module

Layer	Required technology
Frontend	React + Next.js App Router + TypeScript; responsive accessible UI.
Backend	Next.js Route Handlers/API routes; server-side application/domain services.
Database	Supabase PostgreSQL; approved migrations and Row-Level Security.
Authentication	Supabase Auth + server-side RBAC and ownership checks.
AI / safety	Use only the service boundaries applicable to this module; never bypass M4 deterministic safety rules.
Team workflow	Antigravity for assisted development; assigned Git branch; PR review before merge.

5. Frontend pages and components

Safety/triage status

Clear priority label, numeric score where applicable, red-flag details and timestamp.

Alert/queue state

Critical alert state, acknowledgement/dispatch status and safe failure indication.

Doctor override workflow

Original category, proposed final category, mandatory rationale, clinical notes and confirmation step.

Audit review

Authorized audit events with payload hashes; do not permit editing/deleting audit history.

6. Backend APIs and business logic

POST /api/v1/triage/assess

Master-defined endpoint to execute triage; safety pre-check must run before ML.

GET /api/v1/doctor/queue?status=PENDING_REVIEW

Master-defined doctor queue endpoint; enforce doctor/nurse authorization.

POST /api/v1/doctor/override

Master-defined physician review/override endpoint; require rationale and audit record.

Alert dispatcher

Provider-specific SMS/notification APIs are not specified; implement behind isolated adapter and document chosen provider.

7. Database tables and fields

Use the master specification as the source of truth. Do not independently edit shared schemas. Any missing schema/API contract below must be agreed in a reviewed PR before implementation.

triage_assessments

calculated_risk_score; calculated_category; hard_red_flag_triggered; red_flag_details; confidence_score; explainability_payload; status.

triage_reviews

assessment_id FK; doctor_id; original_category; final_category; generated is_overridden; override_reason; clinical_notes; reviewed_at.

triage_audit_log

event_type; user_id; patient_id; ip_address; user_agent; payload; created_at; append-only immutable journal.

8. Input validation and error handling

- Validate on both client and server; client-side checks are not a security boundary.
- Use explicit types, required-field checks, length/range checks, allowed enums and safe parsing for JSON payloads.
- Use the master API success/error envelope consistently; do not return stack traces or PHI in error messages.
- Show clear loading, empty, success, warning and retry/error states; preserve user input safely when a recoverable request fails.
- For this module, apply these specific checks: 100% branch coverage on deterministic red-flag safety engine as required by master spec.; Each of the 12 risk domains has positive, negative and boundary tests.; Red flag bypasses ML and always produces Critical/P1 even if model is unavailable.

9. Security requirements

- Enforce server-side RBAC and patient/record ownership checks on every protected endpoint.
- Use Supabase RLS as defense in depth; never expose service-role keys or GEMINI_API_KEY to browser code.
- Do not log plaintext PHI, tokens, secrets or full sensitive request payloads; use redacted, access-controlled audit events.
- Respect least privilege, admin PHI masking, encryption requirements and immutable audit rules from the master specification.

- Keep the module within decision-support boundaries; no independent diagnosis or prescribing.

10. Integration dependencies with other modules

- M2 supplies vital and symptom vectors.
- M3 prediction is invoked only when no Tier-1 red flag is detected.
- M5 consumes the doctor queue, alert status and override review records.
- M1 provides authenticated role identity; all overrides must be authorized attending-doctor actions.

11. Testing requirements

- 100% branch coverage on deterministic red-flag safety engine as required by master spec.
- Each of the 12 risk domains has positive, negative and boundary tests.
- Red flag bypasses ML and always produces Critical/P1 even if model is unavailable.
- Override without rationale is rejected; unauthorized role is denied; audit entry is append-only.
- API contract tests and full intake-to-override E2E simulation.

12. Expected output and completion criteria

- All master red-flag rules and vital baselines are implemented without unsanctioned threshold changes.
- Critical escalation works even when ML or notification provider fails.
- Override and audit ledger pass authorization and integrity tests.
- 100% red-flag engine branch coverage and integration PR approved.

Antigravity implementation prompt

```
You are implementing MediAI M4: Smart Triage, Safety Rules & Alerts. Read the master project specification, this module specification, README, shared types, existing components, database migrations and API contracts first. Work only on branch feature/m4-smart-triage and keep changes within this module unless a shared-file change is approved. Use React, Next.js App Router, TypeScript, Supabase PostgreSQL/Auth. Reuse shared design tokens and components; do not invent endpoints, database fields, clinical thresholds or tests. Preserve the master safety rules, API envelopes, RBAC/RLS and PHI protections. Before editing, list planned files and assumptions; flag unspecified contracts for team approval. After implementation, run available lint/typecheck/tests, report actual commands/results honestly, and summarize changed files and remaining integration dependencies.
```

Definition of done and handoff checklist

- Feature implemented against approved contract; no speculative schema/API changes.
- Lint, type-check, relevant unit/API/integration tests run and real results reported.
- No secrets or plaintext PHI committed; role access and safe error states verified.
- UI follows shared design tokens, bilingual/accessibility conventions and responsive behavior.
- Commit on feature/m4-smart-triage, push to origin, open Pull Request, attach test evidence and request integration review.